HERMES

Sistema Inteligente de PQRSD

Alcaldía de Medellín

Versión	Evento
1.0.0	**Hackathon 2026**

1. Resumen Ejecutivo

Hermes es una plataforma de gestión de Peticiones, Quejas, Reclamos, Sugerencias y Denuncias (PQRSD) desarrollada para la Alcaldía de Medellín. Su propósito es transformar la interacción entre el ciudadano y la administración distrital mediante inteligencia artificial avanzada y un diseño centrado en el usuario.

El sistema automatiza el ciclo completo de una PQRSD: desde la recepción guiada por un chatbot, pasando por la clasificación semántica y la asignación a la dependencia competente, hasta el seguimiento en tiempo real por parte de los funcionarios. Todo ello garantizando el cumplimiento de los términos legales establecidos en la Ley 1755 de 2015.

Valor diferencial

Hermes reduce el tiempo de clasificación manual de solicitudes ciudadanas de horas a segundos, y ofrece precedentes históricos relevantes para fundamentar respuestas jurídicas mediante búsqueda semántica vectorial (RAG).

2. Características Principales

2.1 Asistente IA Interactivo (Chatbot)

Un chatbot conversacional guía al ciudadano durante la redacción de su solicitud. Al finalizar la conversación, el sistema extrae automáticamente los elementos clave:

- Asunto principal de la solicitud
- Ubicación geográfica mencionada (barrio, comuna, dirección)
- Hechos relevantes y descripción detallada
- Categoría tentativa (petición, queja, reclamo, sugerencia o denuncia)

2.2 Clasificación Automática con IA

Un motor de inteligencia artificial analiza semánticamente el texto de cada solicitud y la asigna a la dependencia municipal competente. Las secretarías soportadas incluyen:

- Secretaría de Movilidad — infraestructura vial y transporte
- Secretaría de Seguridad — orden público y convivencia
- Secretaría de Salud — servicios médicos y saneamiento
- Otras dependencias distritales según la temática detectada

2.3 Búsqueda de Precedentes (RAG)

El sistema implementa Retrieval-Augmented Generation (RAG) sobre una base de datos vectorial construida con pgvector en PostgreSQL. Ante cada nueva solicitud, el motor identifica

los casos históricos más similares y los presenta al funcionario como referencia para acelerar la formulación de la respuesta jurídica.

2.4 Dashboard Administrativo

Panel de gestión completo para los funcionarios, que incluye:

- Visualización estadística de solicitudes por estado, tipo y dependencia
- Mapas de calor (Heatmaps) georreferenciados por comunas y barrios de Medellín
- Tablero Kanban para el seguimiento de estados (Recibida, En revisión, Respondida, Cerrada)
- Filtros avanzados por fecha, tipo de solicitud y dependencia asignada

2.5 Cálculo de Términos de Ley

El sistema calcula automáticamente las fechas de vencimiento de cada solicitud de acuerdo con la Ley 1755 de 2015 (Derecho de Petición). El cálculo incluye la gestión correcta de festivos en Colombia utilizando la librería holidays, garantizando el cumplimiento normativo.

3. Arquitectura del Sistema

3.1 Visión General

Hermes sigue una arquitectura cliente-servidor con separación clara entre el frontend y el backend. La comunicación se realiza a través de una API REST construida con FastAPI. Los datos persisten en una instancia de PostgreSQL gestionada por Supabase, con la extensión pgvector habilitada para las búsquedas semánticas.

3.2 Estructura de Directorios

```
Hermes/
├── app/                    # Backend principal
│   ├── main.py             # Configuración FastAPI, CORS, routers
│   ├── models/             # Modelos SQLModel (ORM + esquema)
│   ├── schemas/            # Esquemas Pydantic para validación
│   ├── routers/            # Endpoints agrupados por dominio
│   ├── services/           # Lógica de negocio e integración IA
│   └── utils/              # Utilidades (festivos, fechas, etc.)
├── frontend-hermes/        # Frontend React + Vite
│   ├── src/
│   │   ├── pages/          # Páginas principales (Chat, Dashboard, Heatmap)
│   │   ├── components/     # Componentes reutilizables
│   │   └── styles/         # Configuración Tailwind CSS 4
│   └── package.json
├── main.py                 # Punto de entrada para Vercel
├── requirements.txt        # Dependencias Python
```

```
├── vercel.json            # Configuración de despliegue
└── .env                   # Variables de entorno (no versionado)
```

3.3 Flujo de una Solicitud PQRSD

Campo	Descripción
1. Recepción	El ciudadano interactúa con el chatbot IA. El asistente extrae asunto, ubicación y hechos.
2. Clasificación	El motor de IA asigna la solicitud a la dependencia competente mediante análisis semántico.
3. Persistencia	La solicitud se almacena en PostgreSQL junto con su embedding vectorial (pgvector).
4. Términos	Se calcula automáticamente la fecha de vencimiento legal según la Ley 1755 de 2015.
5. Gestión	El funcionario recibe la solicitud en el dashboard Kanban y puede ver precedentes similares (RAG).
6. Respuesta	La respuesta se registra y la solicitud pasa al estado 'Cerrada', notificando al ciudadano.

4. Stack Tecnológico

4.1 Backend

Tecnología	Versión / Detalle	Rol en el sistema
FastAPI	0.115.6	Framework principal de la API REST
Uvicorn	0.33.0 (standard)	Servidor ASGI de alto rendimiento
SQLModel	0.0.22	ORM basado en Pydantic + SQLAlchemy
Supabase Python	2.11.0	Cliente BaaS para auth, DB y storage
pydantic-settings	2.7.1	Gestión de configuración y .env
httpx	0.27.0	Cliente HTTP asíncrono para llamadas IA
python-multipart	0.0.9	Soporte para formularios multipart
holidays	0.47	Festivos colombianos para cálculo de términos
python-dotenv	1.0.1	Carga de variables de entorno

Python	3.11+	Lenguaje de programación del backend

4.2 Frontend

Tecnología	Versión / Detalle	Rol en el sistema
React	19	Librería principal de UI
Vite	6.4.2	Bundler y servidor de desarrollo
Tailwind CSS	4	Estilos utilitarios de alto rendimiento
Framer Motion	Última estable	Animaciones y transiciones de UI
Lucide React	Última estable	Iconografía moderna y consistente
TypeScript	Integrado	Tipado estático del proyecto

4.3 Infraestructura y Base de Datos

Tecnología	Versión / Detalle	Rol en el sistema
PostgreSQL	15+ (Supabase)	Base de datos relacional principal
pgvector	Extensión	Búsqueda vectorial para RAG
Supabase	BaaS	Autenticación, storage y gestión de DB
HuggingFace	Inference API	Modelos de embeddings y clasificación
Vercel	v2	Despliegue serverless del backend

5. Configuración e Instalación

5.1 Requisitos Previos

- Python 3.10 o superior
- Node.js 18 o superior
- Cuenta de Supabase con la extensión vector habilitada
- Clave de API de HuggingFace para los modelos de IA

5.2 Variables de Entorno

El archivo .env (no versionado) debe contener las siguientes variables. Consulte el archivo .env.example como referencia:

Variable	Descripción	Ejemplo
SUPABASE_URL	URL del proyecto Supabase	https://xxx.supabase.co
SUPABASE_KEY	Clave anon/service de Supabase	eyJhbGciOiJIUzI1...
HUGGINGFACE_API_KEY	Token de acceso a HuggingFace	hf_xxxxxxxxxxxxxxxx
DATABASE_URL	Cadena de conexión a PostgreSQL	postgresql://user:pass@host/db
ENVIRONMENT	Entorno de ejecución	development / production

5.3 Instalación del Backend

```
# 1. Crear y activar entorno virtual
python -m venv venv
source venv/bin/activate       # Linux / macOS
venv\Scripts\activate           # Windows

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Configurar variables de entorno
cp .env.example .env
# Editar .env con sus credenciales

# 4. Iniciar el servidor de desarrollo
uvicorn app.main:app --reload --port 8000
```

5.4 Instalación del Frontend

```
# 1. Navegar al directorio del frontend
cd frontend-hermes

# 2. Instalar dependencias
npm install

# 3. Iniciar el servidor de desarrollo
npm run dev

# 4. Compilar para producción
npm run build
```

6. Problema Conocido y Resolución

6.1 Error de Build — @vgrid/react-leaflet-heatmap-layer

Error detectado
Rollup failed to resolve import "@vgrid/react-leaflet-heatmap-layer" from src/pages/HeatmapPage.tsx

Durante el proceso de compilación para producción (npm run build), Vite/Rollup no puede resolver el módulo @vgrid/react-leaflet-heatmap-layer porque este paquete no está instalado en el proyecto. A continuación se presentan dos soluciones:

Solución A — Instalar el paquete faltante (recomendada)

```
cd frontend-hermes
npm install @vgrid/react-leaflet-heatmap-layer
npm run build
```

Esta solución mantiene la funcionalidad completa del mapa de calor en la página HeatmapPage.tsx.

Solución B — Externalizar el módulo en Vite

Si se prefiere excluir el mapa de calor del bundle de producción, se puede externalizar el módulo en vite.config.ts:

```
// vite.config.ts
export default defineConfig({
  build: {
    rollupOptions: {
      external: ['@vgrid/react-leaflet-heatmap-layer']
    }
  }
})
```

Nota: con esta opción la página HeatmapPage no funcionará en producción a menos que el paquete se cargue mediante CDN o se sustituya por una implementación alternativa.

7. Despliegue en Vercel

7.1 Configuración vercel.json

El archivo vercel.json configura el despliegue serverless del backend FastAPI en Vercel. La configuración actual enruta las peticiones de la API, la documentación Swagger y la raíz hacia el entry point main.py:

```
{
```

```
  "version": 2,
  "builds": [{ "src": "main.py", "use": "@vercel/python" }],
  "routes": [
    { "src": "/api/(.*)", "dest": "main.py" },
    { "src": "/",         "dest": "main.py" },
    { "src": "/docs",     "dest": "main.py" },
    { "src": "/openapi.json", "dest": "main.py" }
  ]
}
```

7.2 Pasos de Despliegue

1. Instalar Vercel CLI: npm install -g vercel
2. Autenticarse: vercel login
3. Configurar las variables de entorno en el panel de Vercel (Settings > Environment Variables).
4. Desplegar: vercel --prod

8. Referencia de la API

8.1 Documentación Automática

FastAPI genera documentación interactiva de forma automática. Una vez el servidor está en ejecución, los endpoints están disponibles en:

- Swagger UI — http://localhost:8000/docs
- ReDoc — http://localhost:8000/redoc
- Esquema OpenAPI — http://localhost:8000/openapi.json

8.2 Endpoints Principales

Campo	Descripción
POST /api/pqrsd	Crea una nueva solicitud ciudadana (PQRSD) con clasificación automática.
GET /api/pqrsd	Lista todas las solicitudes con filtros de estado, tipo y dependencia.
GET /api/pqrsd/{id}	Obtiene el detalle completo de una solicitud por su identificador.
PATCH /api/pqrsd/{id}	Actualiza el estado o la dependencia asignada de una solicitud.
POST /api/chat	Endpoint de conversación con el chatbot IA para asistir al ciudadano.
GET /api/stats	Retorna estadísticas agregadas para el dashboard administrativo.

GET /api/precedents/{id}	Busca solicitudes históricas similares (RAG) para una solicitud dada.

9. Marco Legal

El sistema Hermes ha sido diseñado para cumplir con la normativa colombiana aplicable a la gestión de solicitudes ciudadanas:

Campo	Descripción
Ley 1755 de 2015	Regula el Derecho Fundamental de Petición. Define los términos de respuesta: 15 días hábiles para peticiones, 10 para quejas y reclamos.
Ley 1581 de 2012	Ley Estatutaria de Protección de Datos Personales. Todos los datos de los ciudadanos son tratados conforme al principio de finalidad y minimización.
Decreto 019 de 2012	Supresión de trámites innecesarios. Hermes reduce la carga administrativa mediante automatización.
CONPES 3920	Política Nacional de Explotación de Datos (Big Data). El análisis estadístico del dashboard se alinea con esta política.

10. Equipo y Créditos

Hermes fue desarrollado durante la Hackathon 2026 organizada para la transformación digital de la Alcaldía de Medellín. El sistema representa el compromiso del equipo con la innovación cívica y la aplicación de inteligencia artificial al servicio del ciudadano.

Licencia y uso

Este proyecto es de carácter académico y fue desarrollado para participar en la Hackathon 2026 de la Alcaldía de Medellín. Todos los derechos sobre el código fuente pertenecen a sus autores.